

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

**Sumit Kumar Mahato
(1BF24CS305)**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Sumit Kumar Mahato(1BF24CS305), who is a Bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Mr.ROHITH VAIDYA K
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write a program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	7
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	9
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	12
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	14
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	16
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	19
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	22
9	Implement Fractional Knapsack using Greedy technique.	26
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	29
11	Implement "N-Queens Problem" using Backtracking.	31

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

LAB PROGRAM 1:

Write a program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#define MAX 100
int queue[MAX], front = -1, rear = -1;
void enqueue(int x) {
    if (rear == MAX - 1) return;
    if (front == -1) front = 0;
    queue[++rear] = x;
}
int dequeue() {
    if (front == -1 || front > rear) return -1;
    return queue[front++];
}
int isEmpty() {
    return (front == -1 || front > rear);
}
int main() {
    int n, i, j;
    int adj[MAX][MAX], indegree[MAX] = {0};
    int topo[MAX], count = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(adj[i][j] == 1) {
                indegree[j]++;
            }
        }
    }
}
```

```

for(i = 0; i < n; i++) {
    if(indegree[i] == 0) {
        enqueue(i);
    }
}
while(!isEmpty()) {
    int v = dequeue();
    topo[count++] = v;
    for(j = 0; j < n; j++) {
        if(adj[v][j] == 1) {
            indegree[j]--;
            if(indegree[j] == 0) {
                enqueue(j);
            }
        }
    }
}
if(count != n) {
    printf("Cycle detected! Topological sort not possible.\n");
} else {
    printf("Topological Order:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", topo[i]);
    }
}
return 0;
}

```

Output:

```

Enter number of vertices: 6
Enter adjacency matrix:
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Topological Order: 0 1 2 3 4 5

Process returned 0 (0x0)   execution time : 37.682 s
Press any key to continue.

```

Problem List<>🔍

Description

Editorial

Solutions

Submissions

207. Course Schedule

Solved

Medium

Topics

Companies

Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]` indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `true`

Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.

18.1K🔒🔖🌟🔔332 Online

Accepted

← All Submissions

Accepted 54 / 54 testcases passed

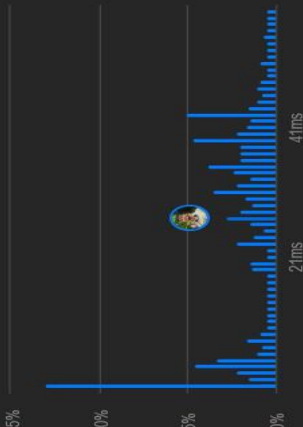
22jtu163h submitted at May 28, 2026 12:25

Runtime

27 msBeats 58.35%

Memory

110.32 MBBeats 16.44%



Code | C

</> Code

C v Auto

Ln 1, Col 1

```
1 bool canFinish(int numCourses, int** prerequisites,
2               int prerequisitesSize, int* prerequisitesColSize) {
3
4     int** graph = (int**)malloc(numCourses * sizeof(int*));
5     int* graphSize = (int*)calloc(numCourses, sizeof(int));
6     int* indegree = (int*)calloc(numCourses, sizeof(int));
7
8     for (int i = 0; i < numCourses; i++) {
9         graph[i] = (int*)malloc(numCourses * sizeof(int));
10    }
```

Testcase

Test Result

Case 1

Case 2

numCourses =

2

prerequisites =

[[1,0]]

</> Source

LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1

int getLargestMobile(int perm[], int dir[], int n) {
    int largestMobile = 0, index = -1;
    for (int i = 0; i < n; i++) {
        int neighbor = i + dir[i];
        if (neighbor >= 0 && neighbor < n && perm[i] > perm[neighbor]) {
            if (perm[i] > largestMobile) {
                largestMobile = perm[i];
                index = i;
            }
        }
    }
    return index;
}

void printPermutation(int perm[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", perm[i]);
    printf("\n");
}

void johnsonTrotter(int n) {
    int perm[n], dir[n];
    for (int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(perm, n);
    while (1) {
        int largestIndex = getLargestMobile(perm, dir, n);
        if (largestIndex == -1) break;
        int swapIndex = largestIndex + dir[largestIndex];
        int temp = perm[largestIndex];
        perm[largestIndex] = perm[swapIndex];
```

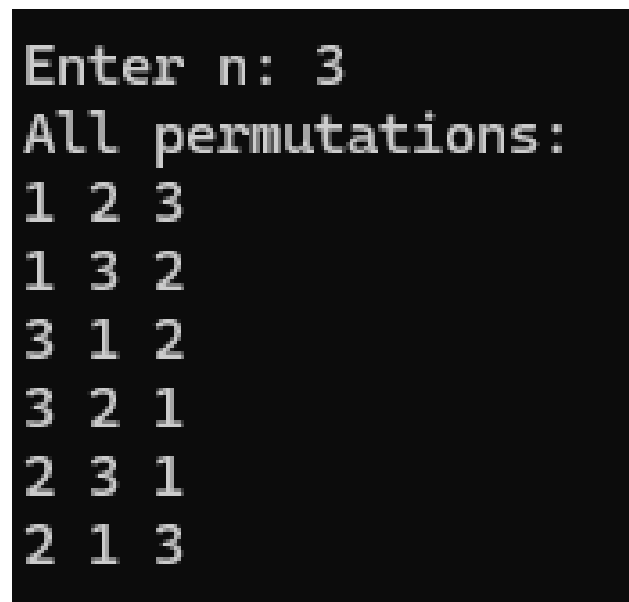
```

    perm[swapIndex] = temp;
    int tempDir = dir[largestIndex];
    dir[largestIndex] = dir[swapIndex];
    dir[swapIndex] = tempDir;
    for (int i = 0; i < n; i++)
        if (perm[i] > perm[swapIndex])
            dir[i] = -dir[i];
    printPermutation(perm, n);
}
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("All permutations:\n");
    johnsonTrotter(n);
    return 0;
}

```

Output:



```

Enter n: 3
All permutations:
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

```


LAB PROGRAM 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 50000
int arr[MAX];
int temp[MAX];
void merge(int l, int mid, int r){
    int i = l, j = mid + 1, k = l;
    while(i <= mid && j <= r){
        if(arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while(i <= mid)
        temp[k++] = arr[i++];
    while(j <= r)
        temp[k++] = arr[j++];
    for(i = l; i <= r; i++)
        arr[i] = temp[i];
}
void mergeSort(int l, int r){
    if(l < r){
        int mid = (l + r) / 2;
        mergeSort(l, mid);
        mergeSort(mid + 1, r);
        merge(l, mid, r);
    }
}
int main(){
    srand(time(NULL));
    printf("Input Size\tTime Taken (seconds)\n");
```

```

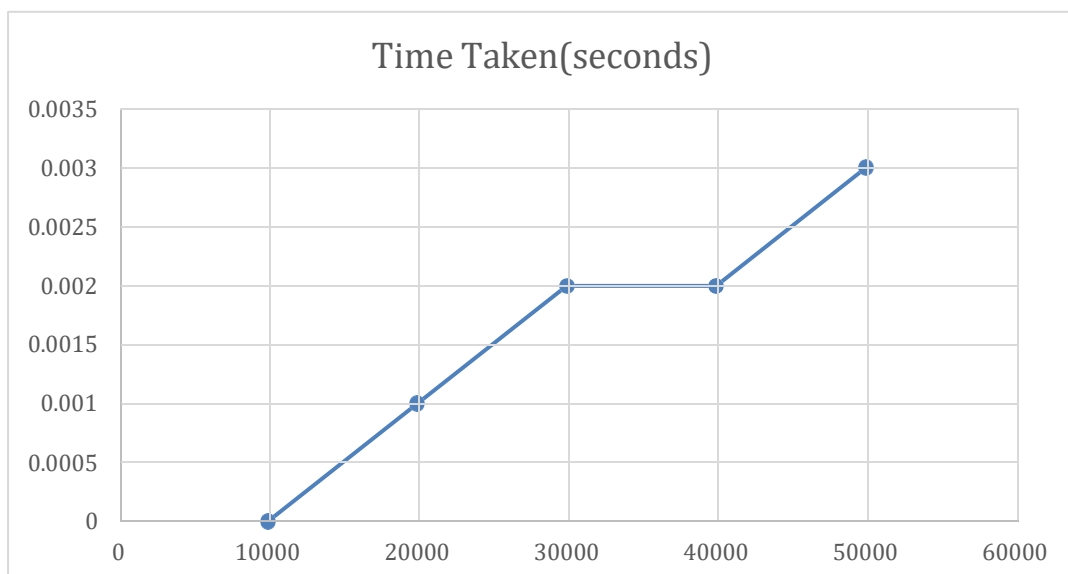
for(int n = 10000; n <= 50000; n += 10000){
    for(int i = 0; i < n; i++){
        arr[i] = rand() % 100000;
        clock_t start = clock();
        mergeSort(0, n - 1);
        clock_t end = clock();
        double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
        printf("%d\t\t%.5f\n", n, time_taken);
    }
}
return 0;
}

```

Output:

Input Size	Time Taken (seconds)
10000	0.00000
20000	0.00100
30000	0.00200
40000	0.00400
50000	0.00500

Time Analysis:



Problem List

Editorial

Solutions

Submissions

912. Sort an Array

Medium

Topics

Companies

Solved

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in `O(n log(n))` time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`

Output: `[1,2,3,5]`

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,1,1,2,0,0]`

Output: `[0,0,1,1,2,5]`

Explanation: Note that the values of `nums` are

7.3K

408

78 Online

Accepted

All Submissions

Accepted 21 / 21 testcases passed

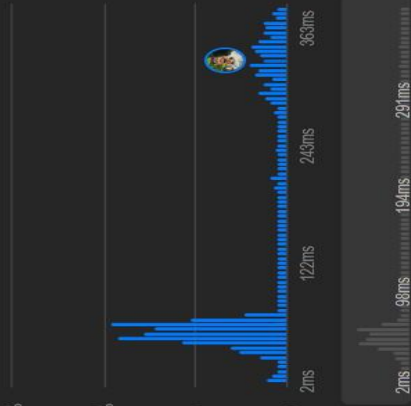
22jui63h submitted at May 28, 2026 12:52

Runtime

332 ms Beats 19.27%

Memory

154.06 MB Beats 6.08%



Code

C

Auto

merge(nums, left, mid, right);

}

}

int* sortArray(int* nums, int numsSize, int* returnSize) {

mergeSort(nums, 0, numsSize - 1);

*returnSize = numsSize;

}

Ln 59, Col 2

Testcase

Test Result

Case 1

Case 2

nums =

[5,2,3,1]

Source

LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 50000
int arr[MAX];
int partition(int low, int high){
    int pivot = arr[high];
    int i = low - 1;
    for(int j = low; j < high; j++){
        if(arr[j] <= pivot){
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
void quickSort(int low, int high){
    if(low < high){
        int pi = partition(low, high);
        quickSort(low, pi - 1);
        quickSort(pi + 1, high);
    }
}
int main(){
    srand(time(NULL));
    printf("Input Size\tTime Taken (seconds)\n");
    for(int n = 10000; n <= 50000; n += 10000){
        for(int i = 0; i < n; i++)
```

```

        arr[i] = rand() % 100000;
    clock_t start = clock();
    quickSort(0, n - 1);
    clock_t end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("%d\t\t%f\n", n, time_taken);
}
return 0;
}

```

Output:

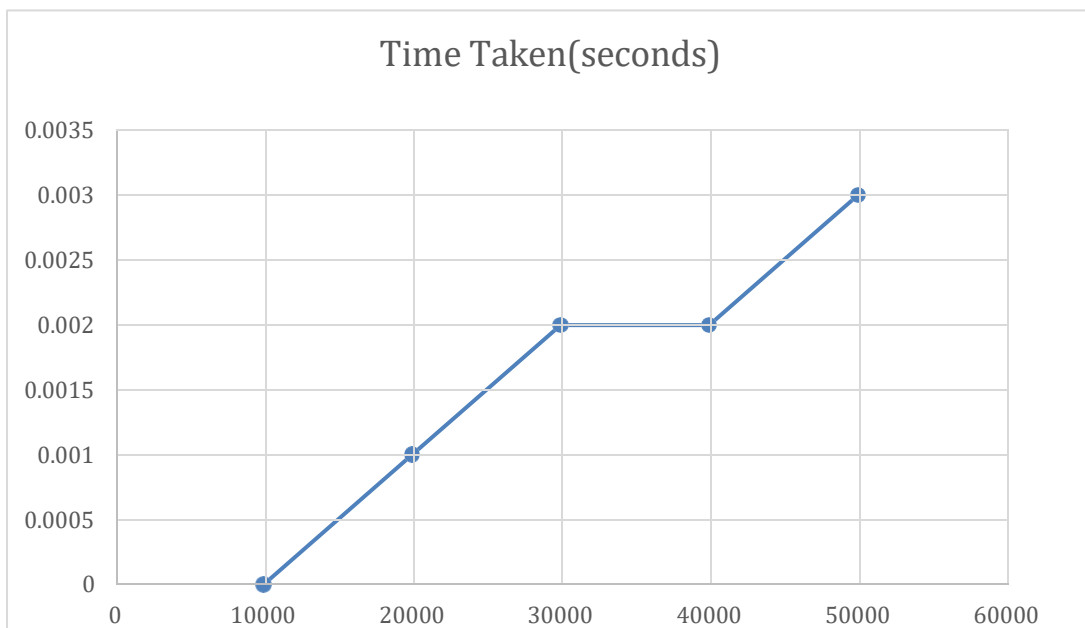
```

Input Size      Time Taken (seconds)
10000           0.000000
20000           0.001000
30000           0.002000
40000           0.002000
50000           0.003000

Process returned 0 (0x0)   execution time : 0.020 s
Press any key to continue.
|

```

Time Analysis:



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>

void heapify(int arr[], int n, int i, int type){
    int extreme = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if (type == 1) {
        if (left < n && arr[left] > arr[extreme])
            extreme = left;
        if (right < n && arr[right] > arr[extreme])
            extreme = right;
    } else {
        if (left < n && arr[left] < arr[extreme])
            extreme = left;
        if (right < n && arr[right] < arr[extreme])
            extreme = right;
    }
    if (extreme != i){
        int temp = arr[i];
        arr[i] = arr[extreme];
        arr[extreme] = temp;
        heapify(arr, n, extreme, type);
    }
}

void buildHeap(int arr[], int n, int type){
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i, type);
}

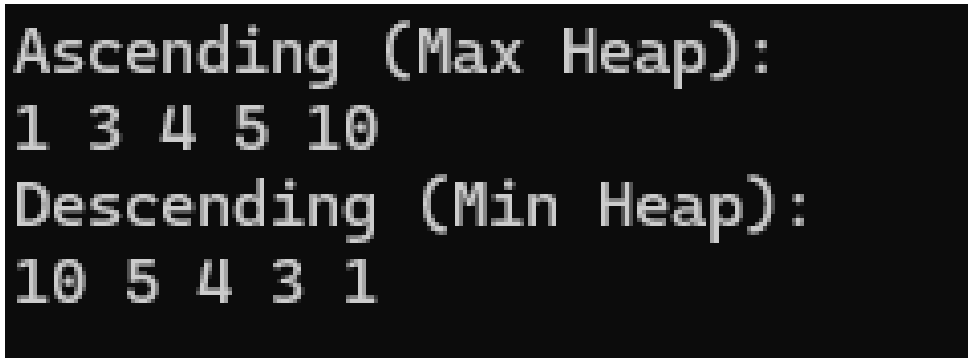
void heapSort(int arr[], int n, int type){
    buildHeap(arr, n, type);
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0, type);
    }
}
```

```

    }
}
void printArray(int arr[], int n){
    for (int i = 0; i < n; i++){
        printf("%d ", arr[i]);
        printf("\n");
    }
}
int main() {
    int arr1[] = {4, 10, 3, 5, 1};
    int arr2[] = {4, 10, 3, 5, 1};
    int n = 5;
    heapSort(arr1, n, 1);
    printf("Ascending (Max Heap):\n");
    printArray(arr1, n);
    heapSort(arr2, n, 0);
    printf("Descending (Min Heap):\n");
    printArray(arr2, n);
    return 0;
}

```

Output:



```

Ascending (Max Heap):
1 3 4 5 10
Descending (Min Heap):
10 5 4 3 1

```

LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int max(int a, int b){
    return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n){
    int i, w;
    int K[n + 1][W + 1];
    for(i = 0; i <= n; i++){
        for(w = 0; w <= W; w++){
            if(i == 0 || w == 0){
                K[i][w] = 0;
            }
            else if(wt[i - 1] <= w){
                K[i][w] = max(
                    val[i - 1] + K[i - 1][w - wt[i - 1]],
                    K[i - 1][w]
                );
            }
            else{
                K[i][w] = K[i - 1][w];
            }
        }
    }
    return K[n][W];
}
```



```

int main(){
    int n, W, i;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int val[n], wt[n];
    for(i = 0; i < n; i++){
        printf("Enter value and weight of item %d: ", i + 1);
        scanf("%d %d", &val[i], &wt[i]);
    }
    printf("Enter capacity of knapsack: ");
    scanf("%d", &W);
    printf("Maximum Profit = %d\n", knapsack(W, wt, val, n));
    return 0;
}

```

Output:

```

Enter number of items: 4
Enter value and weight of item 1: 1      1
Enter value and weight of item 2: 4      3
Enter value and weight of item 3: 5      4
Enter value and weight of item 4: 7      5
Enter capacity of knapsack: 7
Maximum Profit = 9

Process returned 0 (0x0)   execution time : 16.808 s
Press any key to continue.

```

Problem List<>🔍

EditorialSolutionsSubmissions

Solved

494. Target Sum

MediumTopicsCompanies

You are given an integer array `nums` and an integer `target`.

You want to build an **expression** out of `nums` by adding one of the symbols `+` and `-` before each integer in `nums` and then concatenate all the integers.

- For example, if `nums = [2, 1]`, you can add a `+` before `2` and a `-` before `1` and concatenate them to build the expression `"+2-1"`.

Return the number of different **expressions** that you can build, which evaluates to `target`.

Example 1:

Input: `nums = [1,1,1,1,1]`, `target = 3`

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of `nums` be `target` 3.

`-1 + 1 + 1 + 1 + 1 = 3`

`+1 - 1 + 1 + 1 + 1 = 3`

12.4K🔒🔄🔍229🌟🔖🔗🔔76 Online

Accepted

All Submissions

Accepted 142 / 142 testcases passed

22Jul163h submitted at May 28, 2026 14:10

Runtime

0 msBeats 100.00%

Memory

9.12 MBBeats 11.53%

CodeC

Code

CvAuto

```
1 int findTargetSumWays(int* nums, int numsSize, int target) {
2
3     int total = 0;
4
5     for (int i = 0; i < numsSize; i++) {
6         total += nums[i];
7     }
8
9     // Impossible case
10    if ((total + 1) % 2 == 0 || total % 2 == 1 || (total + 1) % 2 == 1) {
11
12    }
13 }
```

SavedLn 33, Col 2

TestcaseTest Result

Case 1Case 2+

nums =

[1,1,1,1,1]

target =

3

</>Source

LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
#define INF 99999
#define MAX 10
void floydwarshall(int n, int graph[MAX][MAX]){
    int dist[MAX][MAX];
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n; j++){
            dist[i][j] = graph[i][j];
        }
    }
    for (int k = 0; k < n; k++){
        for (int i = 0; i < n; i++){
            for (int j = 0; j < n; j++){
                if (dist[i][k] != INF &&
                    dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]){
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    for (int i = 0; i < n; i++){
        if (dist[i][i] < 0) {
            printf("Negative cycle detected\n");
            return;
        }
    }
    printf("\nAll-pairs shortest path matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
    }
}
```

```

    }
    printf("\n");
}
}

int main() {
    int n;
    int graph[MAX][MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (use %d for INF):\n", INF);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    floydwarshall(n, graph);
    return 0;
}

```

Output:

```

Enter number of vertices: 5

Enter adjacency matrix (use 99999 for INF):
0      4      99999  5      8
99999  0      1      99999  6
2      99999  0      3      99999
99999  99999  1      0      2
1      99999  99999  4      0

All-pairs shortest path matrix:
0 4 5 5 7
3 0 1 4 6
2 6 0 3 5
3 7 1 0 2
1 5 5 4 0

```

Problem List

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

Solved

Medium

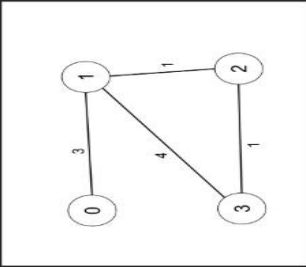
TopicsCompaniesHint

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is **at most** `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities `i` and `j` is equal to the sum of the edges' weights along that path.

Example 1:



```
graph LR
    0 ---|3| 1
    1 ---|1| 2
    1 ---|4| 3
    2 ---|1| 3
```

Input: `n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`

3.6K

227

24 Online

Accepted X

All Submissions

Accepted 54 / 54 testcases passed
22xulw3h submitted at May 28, 2026 15:15

Runtime

13 msBeats 63.16%

Memory

11.84 MBBeats 10.05%



CodeC

```
1 int findTheCity(int n, int** edges, int edgesSize,
2               int* edgesColSize, int distanceThreshold) {
3
4     int INF = 100000000;
5
6     int dist[101][101];
7     // Initialize distance matrix
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < n; j++) {
10             if (i == j)
11                 dist[i][j] = 0;
12             else
13                 dist[i][j] = INF;
14         }
15     }
```

Code

```
1 int findTheCity(int n, int** edges, int edgesSize,
2               int* edgesColSize, int distanceThreshold) {
3
4     int INF = 100000000;
5
6     int dist[101][101];
7     // Initialize distance matrix
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < n; j++) {
10             if (i == j)
11                 dist[i][j] = 0;
12             else
13                 dist[i][j] = INF;
14         }
15     }
```

Testcase

Test Result

Case 1Case 2

n = 4

edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

distanceThreshold = 4

Source

Premium

LAB PROGRAM 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Prim's:

```
#include <stdio.h>
#define MAX 100
#define INF 9999
int main() {
    int n, i, j;
    int cost[MAX][MAX];
    int visited[MAX] = {0};
    int min, u = 0, v = 0;
    int totalCost = 0, edges = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    }
    visited[0] = 1;
    printf("\nEdges in MST:\n");
    while(edges < n - 1) {
        min = INF;
        for(i = 0; i < n; i++) {
            if(visited[i]) {
                for(j = 0; j < n; j++) {
                    if(!visited[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }
    }
```

```

        }
    }
    printf("%d -> %d cost = %d\n", u, v, min);
    totalCost += min;
    visited[v] = 1;
    edges++;
}
printf("\nTotal cost of MST = %d\n", totalCost);
return 0;
}

```

Output:

```

Edge    Weight
0 - 1    10
3 - 2     4
0 - 3     5
Total cost of MST = 19

Process returned 0 (0x0)   execution time : 0.007 s
Press any key to continue.
|

```

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Kruskal's:

```
#include <stdio.h>
#define MAX 100
int parent[MAX];
int find(int i) {
    while(parent[i] != i)
        i = parent[i];
    return i;
}
void unionSet(int i, int j) {
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}

int main() {
    int n, i, j, ne = 0;
    int cost[MAX][MAX];
    int min, u, v, a, b;
    int totalCost = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = 9999;
        }
    }
    for(i = 0; i < n; i++)
        parent[i] = i;
```



```

printf("\nEdges in MST:\n");

while(ne < n - 1) {
    min = 9999;
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(cost[i][j] < min) {
                min = cost[i][j];
                u = a = i;
                v = b = j;
            }
        }
    }
    if(find(a) != find(b)) {
        printf("%d -> %d cost = %d\n", u, v, min);
        totalCost += min;
        unionSet(a, b);
        ne++;
    }
    cost[u][v] = cost[v][u] = 9999;
}
printf("\nTotal cost of MST = %d\n", totalCost);
return 0;
}

```

Output:

```

Edge    Weight
2 - 3    4
0 - 3    5
0 - 1   10
Total cost of MST = 19

Process returned 0 (0x0)    execution time : 0.008 s
Press any key to continue.
|

```

LAB PROGRAM 9:

Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>

struct Item {
    int weight;
    int value;
    float ratio;
};

void swap(struct Item *a, struct Item *b) {
    struct Item temp = *a;
    *a = *b;
    *b = temp;
}

void sortItems(struct Item items[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (items[j].ratio < items[j + 1].ratio) {
                swap(&items[j], &items[j + 1]);
            }
        }
    }
}

float fractionalKnapsack(int capacity, struct Item items[], int n) {
    sortItems(items, n);
    float totalValue = 0.0;
    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            capacity -= items[i].weight;
            totalValue += items[i].value;
        }
        else {
            totalValue += items[i].value * ((float)capacity / items[i].weight);
            break;
        }
    }
    return totalValue;
}
```

```

}
int main() {
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item items[n];
    for (int i = 0; i < n; i++) {
        printf("Enter value and weight of item %d: ", i + 1);
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    float maxVal = fractionalKnapsack(capacity, items, n);
    printf("Maximum value in Knapsack = %.2f\n", maxVal);
    return 0;
}

```

Output:

```

Enter number of items: 3
Enter value and weight of item 1: 60    10
Enter value and weight of item 2: 100   20
Enter value and weight of item 3: 120   30
Enter knapsack capacity: 50
Maximum value in Knapsack = 240.00

Process returned 0 (0x0)    execution time : 29.800 s
Press any key to continue.

```

Problem List

DescriptionEditorialSolutionsSubmissions

Solved

316. Remove Duplicate Letters

MediumTopicsCompaniesHint

Given a string `s`, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the **smallest in lexicographical order** among all possible results.

Example 1:

Input: `s = "bcabc"`
Output: `"abc"`

Example 2:

Input: `s = "cbacdcbc"`
Output: `"acdb"`

Constraints:

- `1 <= s.length <= 104`
- `s` consists of lowercase English letters.

Note: This question is the same as 1081: <https://leetcode.com/problems/smallest-subsequence-of-distinct-characters/>

9.3K17217 Online

Accepted

All Submissions

Accepted 280 / 290 testcases passed

zzju16ish submitted at May 29, 2026 21:01

Runtime

1 ms | Beats 9.79%

Memory

9.04 MB | Beats 24.68%

Analysis

Solution

100%

50%

0%

1ms

2ms

3ms

4ms

1ms

2ms

3ms

4ms

Code

C

```
1 char* removeDuplicateLetters(char* s) {
2
3     int freq[26] = {0};
4     int visited[26] = {0};
5
6     for (int i = 0; s[i] != '\0'; i++) {
7         freq[s[i] - 'a']++;
8     }
9
10    int n = strlen(s);
11
12    char* stack = (char*)malloc((n + 1) * sizeof(char));
13    int top = -1;
14
```

Testcase

Test Result

Case 1

Case 2

s =

"bcabc"

</> Source

</> Code

CvAuto

```
1 char* removeDuplicateLetters(char* s) {
2
3     int freq[26] = {0};
4     int visited[26] = {0};
5
6     for (int i = 0; s[i] != '\0'; i++) {
7         freq[s[i] - 'a']++;
8     }
9
10    int n = strlen(s);
11
12    char* stack = (char*)malloc((n + 1) * sizeof(char));
13    int top = -1;
14
```

Saved

Ln 39, Col 2

LAB PROGRAM 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
#include <limits.h>
#define V 5

int minDistance(int dist[], int visited[]) {
    int min = INT_MAX, minIndex;
    for (int v = 0; v < V; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int visited[V];
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        visited[u] = 1;
        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("Vertex\tDistance from Source\n");
```

```

    for (int i = 0; i < V; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
}

int main() {
    int graph[V][V] = {
        {0, 10, 0, 30, 100},
        {10, 0, 50, 0, 0},
        {0, 50, 0, 20, 10},
        {30, 0, 20, 0, 60},
        {100, 0, 10, 60, 0}
    };
    int source = 0;
    dijkstra(graph, source);
    return 0;
}

```

Output:

```

Enter number of vertices: 4
Enter adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Enter starting vertex: 0
Vertex    Distance from Source
0         0
1         2
2         5
3         6

```

LAB PROGRAM 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdlib.h>
int N;
int *board;
int isSafe(int row, int col) { for
    (int i = 0; i < row; i++) {
        if (board[i] == col ||
            abs(board[i] - col) == abs(i - row)) {
            return 0;
        }
    }
    return 1;
}
void printSolution() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
    printf("\n");
}
void solveNQueens(int row) {
    if (row == N) {
        printSolution();
        return;
    }
    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
```

```

        solveNQueens(row + 1);
    }
}

int main() {
    printf("Enter the value of N: ");
    scanf("%d", &N);
    board = (int *)malloc(N * sizeof(int));
    printf("\nSolutions for %d-Queens Problem:\n\n", N);
    solveNQueens(0);
    free(board);
    return 0;
}

```

Output:

```

Enter the value of N: 4

Solutions for 4-Queens Problem:

. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .

Process returned 0 (0x0)   execution time : 3.483 s
Press any key to continue.

```